Enabling Intelligent Collaboration in Architectural Programming: A Hierarchical Multi-Agent Reinforcement Learning Framework for Mega Transportation Projects

Shitao Jin, Ph.D.[1]; Yuan Deng[2]; and Huijun Tu, Ph.D.[3]

[1]College of Architecture and Urban Planning, Tongji Univ., Shanghai, China (corresponding author). Email: shitaojin@tongji.edu.cn
[2]College of Architecture and Urban Planning, Tongji Univ., Shanghai, China. Email: 2310364@tongji.edu.cn
[3]Professor, College of Architecture and Urban Planning, Tongji Univ., Shanghai, China. Email: 04184@tongji.edu.cn

ABSTRACT

In mega transportation projects (MTPs), architectural programming serves as the starting point of the project lifecycle, and its quality directly influences subsequent stages of design, construction, and operation. However, architectural programming often faces challenges such as multi-stakeholder collaboration, multi-objective conflicts, and uncertainties in the external environment. This study proposes a hierarchical multi-agent reinforcement learning framework, which integrates multi-agent proximal policy optimization (MAPPO), counterfactual regret minimization (CFR), and hierarchical reinforcement learning (HRL) to enhance intelligent collaborative decision-making during the architectural programming stage of MTPs. Using empirical data from the Shanghai East Railway Station project, a simulation environment is constructed to validate the framework's feasibility and effectiveness in improving solution quality, decision-making efficiency, and conflict minimization. The study not only offers methodological support for the intelligent transformation of architectural programming but also provides a transferable technical pathway for multi-stakeholder collaboration optimization in complex systems.

INTRODUCTION

In mega transportation projects (MTPs), architectural programming serves as the foundational stage across the entire project lifecycle. Its rationality and scientific rigor directly influence the pathways and quality of subsequent design coordination, construction organization, and operational performance (Jin et al. 2024). The core task of architectural programming is to systematically define the project's functional orientation, spatial configuration, process layout, and service capacity, while balancing constraints such as limited time, budget, and resources (Pena et al. 2012). However, due to the high public visibility, complex regulatory requirements, and the coexistence of sustainability and environmental protection goals in MTPs, architectural programming must integrate diverse expectations and interests from multiple stakeholders, including governments, designers, constructors, operators, and users (Mok et al. 2015). The diversity of stakeholder interests, potential conflicts in objectives, and limitations in information-sharing mechanisms often lead to deviations from initial programming goals. These deviations can trigger design revisions, construction rework, or even social controversies in later stages, thereby increasing overall project risks and uncertainty (Hu et al. 2015).

In recent years, the rapid development of multi-agent reinforcement learning (MARL) has provided a promising methodological foundation for addressing complex, multi-actor

collaborative decision-making problems (Gronauer et al., 2022). MARL enables the simulation of interactions, learning, and adaptive behavior among autonomous agents in dynamic game environments. It is particularly well-suited for scenarios characterized by partial observability and highly coupled system states, where it optimizes both individual and collective strategies with strong flexibility and adaptability (Gerber et al. 2017). Specifically, in contexts involving multi-stakeholder conflict resolution and collective intelligence optimization, MARL leverages continuous interaction and feedback mechanisms to iteratively refine policy strategies, thus showing theoretical potential for enabling intelligent evolution in architectural programming (Veloso et al. 2023). Nevertheless, the application of MARL approaches to MTP-related architectural programming remains limited due to several critical shortcomings: (1) Lack of effective hierarchical structure modeling, resulting in an overly large policy space and inefficient search processes; (2) Insufficient support for dynamic behavioral correction and game-theoretic feedback among stakeholders, which often leads to suboptimal equilibria and prevents the emergence of forward-looking collaborative solutions.

To address these challenges, this study proposes a hierarchical MARL framework for architectural programming in MTPs. The research objectives are as follows: (1) To develop an integrated framework that combines multi-agent proximal policy optimization (MAPPO), counterfactual regret minimization (CFR), and hierarchical reinforcement learning (HRL) to enable coherent task decomposition and collaborative optimization; (2) To construct a stakeholder behavior modeling mechanism that facilitates continuous policy refinement under partial observability and high uncertainty; (3) To design and validate the proposed framework in real-world MTP scenarios, thereby enhancing the overall quality of architectural programming schemes, decision-making efficiency, and conflict mitigation.

STATE OF THE ART

In the context of MTPs, the complexity of architectural programming is significantly amplified. This is not only due to the need to accommodate massive passenger flows and deeply integrated multimodal transportation systems, but also due to the involvement of multilayered, interdisciplinary stakeholder groups, whose objectives often conflict and require dynamic trade-offs (Li et al. 2019). To address these multidimensional complexities, a growing body of research has focused on advancing the methodologies and technical tools for architectural programming (Table 1). Despite substantial progress in areas such as digitization, information integration, multi-criteria decision analysis, and value-based evaluation, few studies have thoroughly addressed the challenges of multi-agent collaboration under dynamic game conditions and real-time learning—as highlighted in Table 1. Most existing research remains focused on developing static, one-off decision-support tools tailored to single stakeholders or design teams, with limited attention to how diverse actors can collaborate effectively under conditions of incomplete information, conflicting preferences, and evolving decision landscapes (Bansal 2021). Achieving intelligent collaborative architectural programming calls for the development of mechanisms that enable continuous learning and strategic adaptation among multiple stakeholders (Erkul et al. 2020).

In recent years, significant advances in artificial intelligence algorithms and the widespread availability of computational resources have propelled the development of reinforcement learning techniques. Among them, MARL aims to address scenarios involving interactions, cooperation, and competition among multiple autonomous agents, enabling each agent to simultaneously learn and optimize its own policy within shared or conflicting environments (Oroojlooy et al. 2023).

Furthermore, MARL has demonstrated substantial potential in optimizing collective intelligence under multi-objective conflict scenarios, as it enables agents to iteratively refine strategies through simulated repeated games and policy interactions—eventually converging toward more efficient and equitable equilibria (Keith et al. 2021). However, despite the promising capabilities of MARL in multi-agent environments, its direct applicability to the highly hierarchical, strongly interdependent decision-making landscape of MTP architectural programming remains limited (Ivanov et al. 2021). In MTPs, architectural programming typically spans multi-stage, multi-scale, and interdisciplinary tasks, where the decision-making process exhibits a pronounced hierarchical structure and tightly coupled logic (Jin 2025).

Table 1. Previous research on architectural programming in MTPs.

Researches	Research Scope		Methodology			Number of Stakeholders	Development Type
	Theory	Practice	Heuristic	Simulation	Optimization		
[1]	●		●	●		Single	Quantitative
[2]		●	●		●	Multi	Quantitative
[3]	●	●		●	●	Single	Quantitative
[4]	●	●	●			Single	Qualitative
[5]	●		●		●	Single	Qualitative
[6]	●			●		Multi	Quantitative
[7]		●		●	●	Single	Quantitative
[8]	●		●			Single	Quantitative
[9]	●		●		●	Single	Qualitative
[10]		●	●			Single	Qualitative
[11]		●	●		●	Single	Quantitative

References: [1]: Huang et al. (2025); [2]: Tu and Jin (2024); [3]: Platt et al. (2023); [4]: Milovanovic et al. (2023); [5]: Trajkovic et al. (2021); [6]: Ma et al. (2021); [7]: Berawi et al. (2021); [8]: Zhuang et al. (2023); [9]: Pyburn (2017); [10]: Deng and Poon (2013); [11]: Kalayci and Ozdemir (2021).

METHODOLOGY

The methodological framework proposed in this study is illustrated in Figure 1. This framework maps the architectural programming process onto the logic of the MARL algorithm. First, project information is transformed into an environmental state space. Second, agent attributes and actions are constructed based on stakeholder preferences. Third, policy iteration and conflict resolution are driven by a multi-level dynamic game mechanism. Finally, the framework yields a converged optimal policy network, providing quantitative guidance for subsequent design phases.

MAPPO Learning Framework. This study employs the MAPPO algorithm to construct the core collaborative decision-making mechanism for the architectural programming of MTPs. Based on the Centralized Training with Decentralized Execution (CTDE) architecture, this algorithm addresses the challenge where multiple stakeholders with local perspectives fail to achieve global optima (Li et al., 2021).

During the centralized training phase, the system introduces a global Critic. This Critic utilizes the global state S_t at time t and the joint actions A_t of all agents to update the joint value function $Q^{joint}(S_t, A_t)$. This function evaluates the contribution of multi-party joint decisions to the overall programming performance and serves as an advantage function to guide the policy updates of each agent. Consequently, individual decisions are explicitly constrained by system-level objectives.

During the decentralized execution phase, each agent i makes independent decisions utilizing its policy network $\pi_i(a_{i,t} \mid o_{i,t})$, based solely on its local observation $o_{i,t} \subset S_t$. Agents are not required to share policy parameters or explicitly transmit beliefs; instead, relying on the global value guidance internalized during the training phase, they spontaneously form implicit collaborative relationships through dynamic interactions. To ensure the stability of policy iteration, MAPPO adopts the clipped objective function from PPO, as shown in Equation (1):

$$L(\theta) = \mathbb{E}_t \left[\min \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t,\ clip \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}, 1-\epsilon, 1+\epsilon \right) \hat{A}_t \right) \right] \quad (1)$$

Where $\hat{A}_t$ represents the advantage function estimate, and ϵ denotes the clipping threshold used to limit the magnitude of policy updates.

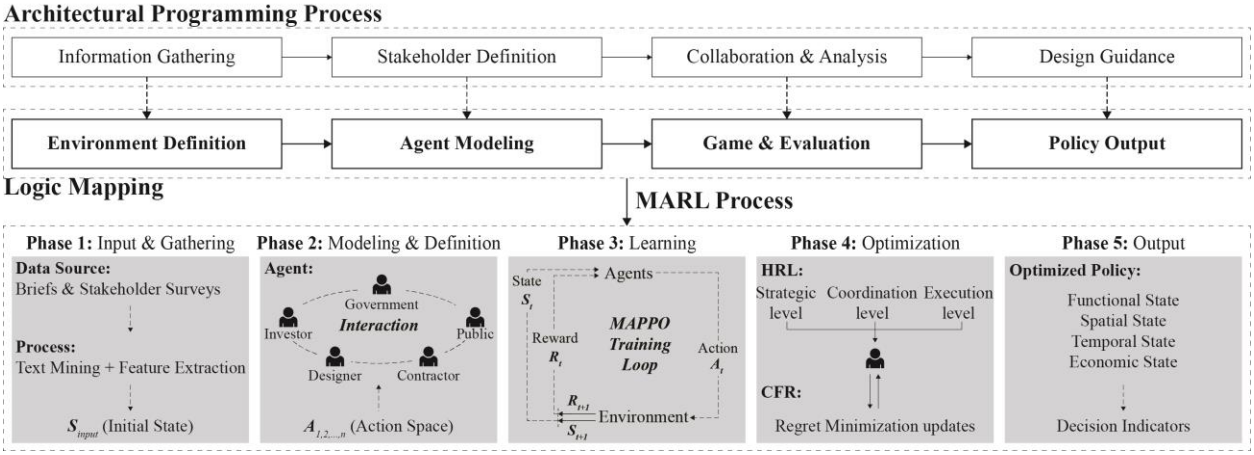

Figure 1. The proposed methodological framework.

To establish the mapping between the architectural programming context and the algorithm, this study further constructs the state space S, action space A, and their interaction relationships, as presented in Table 2. The state space characterizes the architectural programming scenario within which multi-party collaboration occurs. It encompasses five categories of variables: function, space, temporal, stakeholder, and external environment. These variables correspond to core elements in the information gathering phase: functional positioning, spatial organization, implementation schedule, balancing of stakeholder demands, and policy/public constraints, respectively. Correspondingly, the action space describes the decision-making behaviors executable by participating entities during the programming process. It includes scheme selection, resource allocation, priority adjustment, and information exchange, which correspond to key operations in architectural programming: scheme comparison, resource coordination, goal trade-offs, and multi-party communication. These actions modify the corresponding state variables and influence the joint reward function, gradually inducing the evolution of stable collaborative behavioral patterns through multiple rounds of interaction.

CFR-Embedded Reward Structure. To transform abstract MTP programming requirements into computable optimization objectives, this study constructs a feature-reward mapping mechanism, as illustrated in Table 3. The total reward function $R_i(t)$ of the system is formulated

as a linear weighted combination of local objective achievement, global collaboration gain, and conflict penalty, formally defined as:

$$R_i(t) = \omega_1 \cdot r_i^{local}(t) + \omega_2 \cdot r^{collab}(t) - \omega_3 \cdot r^{conflict}(t) \quad (2)$$

Specifically, the baseline proportions of the weighting coefficients $\{\omega_1, \omega_2, \omega_3\}$ are determined during the initialization phase via expert scoring and the Analytic Hierarchy Process (AHP). Subsequently, these coefficients undergo dynamic fine-tuning based on the Pareto frontier during the training process. This ensures adaptation to varying collaborative priorities across different stages and prevents any single objective from dominating the optimization process.

Table 2. Definition of State-Action Spaces and Interaction Dynamics.

Action Space	Target State Space	Interaction Mechanism
Scheme Selection a_{select}	$S_{function}$ / S_{space}	Defines functional types and spatial forms, triggering capacity updates.
Resource Allocation $a_{allocate}$	S_{space} / $S_{temporal}$	Consumes land/budget and determines construction sequencing.
Priority Adjustment $a_{prioritize}$	$S_{stakeholder}$	Re-weights objective preferences to mitigate internal goal conflicts.
Info. Communication $a_{communicate}$	$S_{stakeholder}$ / $S_{external}$	Shares constraints to reduce uncertainty and align with external norms.

The local reward r_i^{local} is directly mapped from the functional requirement state $S_{function}$ and the temporal state $S_{temporal}$, serving to quantify the deviation between the individual agent's programming metrics and the target values. The conflict penalty $r^{conflict}$ is derived from physical overlaps within the spatial state S_{space} and the objective conflict matrix within the stakeholder state $S_{stakeholder}$. This mechanism compels agents to avoid hard constraints regarding resources and interests during decision-making. Meanwhile, the collaboration reward r^{collab} is measured by the marginal gain of the global value function Q^{joint}, incentivizing information exchange and complementary decision-making.

Table 3. Feature-to-Reward Mapping Mechanism.

Reward	Feature Basis	Calculation Logic
$r_i^{local}(t)$	$S_{function}$ (Area/Capacity)	Deviation between state and individual target
	$S_{temporal}$ (Milestones)	Milestone completion accuracy
$r^{collab}(t)$	Global Value Function Q^{joint}	Marginal increment ΔQ^{joint}
$r^{conflict}(t)$	S_{space} (Spatial Overlap)	Quantified by spatial conflict matrix
	$S_{stakeholder}$ (Conflict Weights)	Weighted goal contradiction index

Building upon the reward function, to address policy oscillation caused by information asymmetry and strategic gaming of interests in MTP scenarios, this study embeds the CFR mechanism into the MAPPO framework via a linear weighting approach, thereby enhancing the stability of policy evolution. CFR guides policy updates by calculating the "counterfactual regret

value," defined as the difference in potential payoffs an agent could have achieved had it selected an alternative action in a historical state (Brown et al., 2019). For agent i, the cumulative regret at step t is updated using the following formula:

$$R_i^{(t)}(a) = R_i^{(t-1)}(a) + \left(u_i(a, s_t) - u_i(\sigma_t, s_t)\right) \quad (3)$$

Where $u_i(a, s_t)$ represents the actual action payoff calculated based on the aforementioned reward function, and $u_i(\sigma_t, s_t)$ denotes the expected payoff under the current policy. Accordingly, the agent iteratively updates the action probability distribution for the subsequent stage employing a non-negative normalization method:

$$\pi_i^{(t+1)}(a) = \frac{\max\left(R_i^{(t)}(a), 0\right)}{\sum_{a'} \max\left(R_i^{(t)}(a'), 0\right)} \quad (4)$$

When all action regret values are negative, the system retains the original uniform probability distribution to maintain necessary exploration, thus preventing premature convergence to suboptimal policies.

Hierarchical Decision-making. In real-world programming scenarios for MTPs, the diversity of participating agents, pronounced information heterogeneity, and the structural asymmetry in roles, such as disparities in resource control, value preferences, and task granularity, naturally lead to hierarchical dependencies. Therefore, this study incorporates an HRL module atop the MAPPO framework, enabling structured decomposition of tasks, policy alignment across layers, and intention propagation. In this module, typical agents in MTPs are grouped into three hierarchical tiers according to their functional weight and role characteristics: Upper layer: responsible for setting overall objectives and macro-level adjustments; Middle layer: mediates resource allocation and strategy adaptation; Lower layer: performs concrete actions and operations. Table 4 summarizes the definition of state and action subspaces across layers:

Table 4. Definition of State and Action Subspaces across Hierarchical Agent Roles.

Layer	Agent Roles	Perceived Variable Scope	Controlled Behavior Types
Upper	Government, Public	Functional needs, external environment, key time nodes, public preferences	Goal setting, value prioritization, policy interventions
Middle	Investors	Functional status, spatial status, resource allocation, conflict mapping	Resource adjustment, scheme compatibility, prioritization
Lower	Designers, Contractors	Blueprint conditions, parcel layout, design codes, labor & progress	Design decisions, construction sequencing, strategic responses

During policy execution, the HRL module forms a dynamic coupling loop through bidirectional information flows: goal propagation downward and feedback upward. Specifically, the upper layer generates high-level goals or task conditions g^u, which are passed to the middle layer as input. The middle layer further generates sub-goals g^m, which condition the policies at the lower layer. Based on these sub-goals, the lower layer executes concrete actions a^l. Meanwhile, each layer sends its state changes and performance signals, enabling higher layers to compute corresponding rewards and adapt their policies accordingly. This two-way communication is formalized as:

$$\begin{cases} g_t^{(l+1)} = f^l(s_t^l, a_t^l), \\ r_t^l = \psi^l(s_t^{(l+1)}, a_t^{(l+1)}, \Delta s_t^{(l+1)}) \end{cases}, \forall l \in \{upper, middle\} \quad (5)$$

Here, f^l is the goal generation function at layer l, mapping the current state and action to the next layer's input goal. ψ^l computes the reward for layer l based on the lower layer's feedback. The state increment $\Delta s_t^{(l+1)} = s_{t+1}^{(l+1)} - s_t^{(l+1)}$ reflects the effectiveness of the lower layer in responding to the assigned task.

CASE STUDY

This study selects the Shanghai East Railway Station as a real-world case to validate the adaptability and effectiveness of the proposed MARL framework (Figure 2). As a core component of the Pudong Integrated Transport Hub, the project covers a total gross floor area of approximately 1.3 million square meters, including a high-speed railway station building area of about 160,000 square meters, with completion scheduled for 2027 (SMPG, 2025).

Figure 2. Location and design of the Shanghai East Railway Station.

Data collection for this study involved two approaches: (1) Systematic text mining of the programming briefs and design guidelines for the project to identify core stakeholders and project constraint boundaries; (2) Questionnaire surveys to obtain the decision-making preferences of various participating entities across five programming dimensions: traffic efficiency, functional integration, green and energy-efficient performance, operational returns, and public experience. A total of 150 questionnaires were distributed, with 128 valid responses retrieved (an effective rate of 85.3%). All items were scored using a five-point Likert scale. Based on the survey data, the Entropy Weight Method combined with the AHP was employed to calculate the preference weights of each entity across different dimensions. Specific values are detailed in Table 5.

The simulation environment was constructed based on the PyTorch framework. Key parameters for model training were determined through three rounds of preliminary experiments and grid search (Table 6). Furthermore, all policy and value networks utilized fully connected feedforward neural networks (MLP), comprising three hidden layers with 128 neurons per layer, employing ReLU as the activation function. The output layers were connected to the Softmax function and a linear activation function, respectively, to estimate discrete action probabilities and state values.

Table 5. Stakeholder Classification and Attribute Modeling.

Agent	Stakeholders	Behavioral Traits	Goal Preferences		Weights
Government	Pudong Planning Bureau, Transport Commission	Planning authority, macro control	G_1	Urban integration	0.5
			G_2	Land use efficiency	0.3
			G_3	Regulatory compliance	0.2
Investors	China Railway Group, Eastern Hub Co.	Risk-sensitive, cost-driven	I_1	Return on investment	0.5
			I_2	Construction period	0.3
			I_3	Long-term asset value	0.2
Designers	Municipal Design Institutes, Architecture Firms	Solution-driven, multi-goal	D_1	Functional integration	0.5
			D_2	Aesthetic alignment	0.3
			D_3	Engineering feasibility	0.2
Contractors	CRCC, General Contractors	Time-constrained, resource-limited	C_1	Construction feasibility	0.6
			C_2	Cost control	0.2
			C_3	Process compatibility	0.2
Public	Residents, Passengers, NGOs	User-oriented, feedback-driven	P_1	Transfer efficiency	0.5
			P_2	Environmental quality	0.3
			P_3	User comfort	0.2

Table 6. Experimental Hyperparameters and Model Configuration.

Category	Parameter	Value / Description
Optimization	Optimizer	Adam ($\beta_1 = 0.9, \beta_2 = 0.999$)
	Learning Rate (LR)	3×10^{-4} (Actor & Critic)
	Batch Size	128
	Training Episodes	150,000
MAPPO Algorithm	Discount Factor (γ)	0.99
	Clip Parameter (ϵ)	0.2
	GAE Parameter (λ)	0.95
Specific Modules	CFR Update Frequency	Every 10 episodes
	HRL Goal Interval	Every 50 time steps
	Reward Weights ($\omega_1, \omega_2, \omega_3$)	0.4: 0.35: 0.25

RESULTS

Figure 3 illustrates the training performance of the proposed MARL framework. Overall, the global reward exhibited high volatility during the first 30,000 episodes, reflecting the irrational gaming among multi-agents during the initial phase of strategic exploration. Entering the interval between 40,000 and 80,000 episodes, as the CFR mechanism corrected these irrational behaviors, the agents gradually formed stable collaborative strategies. Consequently, reward levels climbed steadily and converged around 100,000 episodes.

Specifically, the evolution of local rewards revealed the underlying collaborative mechanism. The intense fluctuations of the government agent in the early stages reflected the conflict between rigid constraints and multi-dimensional objectives. However, as upper-layer strategies reshaped resource boundaries, the competitive relationship between the designer and investor agents transformed into a positive-sum game, demonstrating a synchronous rising trend toward

equilibrium. Furthermore, the rapid decline of the loss function and the steady convergence of behavioral entropy verified that the model, under multi-scale learning rate control, effectively avoids local optima, ultimately generating strategies that possess both determinism and robustness.

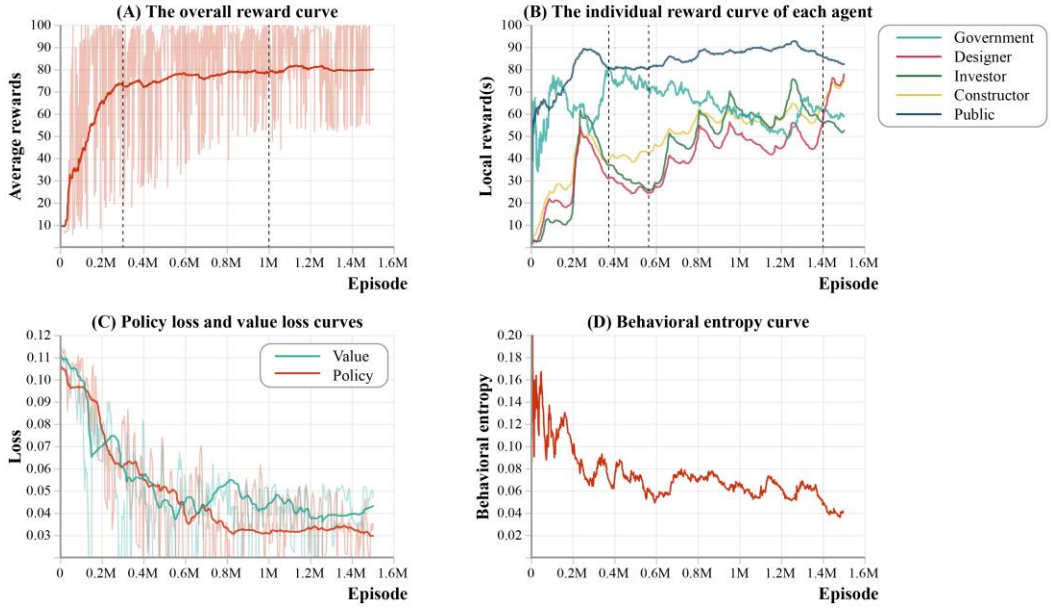

Figure 3. Reward, loss, and behavioral entropy curve.

Based on the converged policy network, this study extracted the final state vectors and calculated the normalized scores for 15 key programming indicators (Table 7). Compared to the baseline scheme, MARL significantly improved Urban Integration (G_1) and Land Use Efficiency (G_2), with increases of 28.0% and 18.5%, respectively. This indicates that agents tend to adopt high-density mixed-use strategies to respond to macro-planning demands. Meanwhile, Functional Integration (D_1) and Transfer Efficiency (P_1) saw substantial improvements, with the latter recording the largest increase (41.5%). This suggests that the model learned a vertical transportation configuration that balances commercial value with circulation efficiency under multiple constraints. In addition, Process Compatibility (C_3) and Compliance (G_3) both achieved positive improvements, rising by 23.8% and 22.0%, respectively, confirming that the CFR mechanism effectively mitigates the risks of deadlocks and violations among multiple stakeholders. Finally, indicators such as Investment Return (I_1) and Long-term Asset Value (I_3) became balanced and compatible after optimization, indicating that a systemic equilibrium between short-term cost control and long-term value operation was achieved among stakeholders.

DISCUSSION AND CONCLUSION

To address the complexity characterizing the architectural programming of MTPs, this study establishes a collaborative decision-making framework integrating MAPPO, CFR, and HRL, and validates its effectiveness through the case of the Shanghai East Railway Station. The findings indicate that the framework not only effectively converges to Pareto improvement solutions but also generates programming strategies that align highly with real-world solutions. This alignment is essentially the projection of multiple agents in their pursuit of a Nash equilibrium. Specifically,

to find the optimal solution between functional integration and investment returns, the designer agent spontaneously adopted a strategy of clustering high-frequency commercial functions around the transit core. Meanwhile, the public agent's extreme pursuit of transfer efficiency forced the spatial layout to abandon traditional horizontal sprawl in favor of vertical stacking and integration. This "multi-level traffic organization" is an emergent morphology arising from the algorithm's effort to maximize urban integration within the boundaries of feasibility while handling high-dimensional conflicts. This demonstrates that AI-driven programming can transcend the linear thinking inherent in heuristics to discover non-linear optimal solutions within complex systems.

Table 7. Comparison of Key Programming Indicators (Normalized).

	Indicators	Pretraining State	Optimization Result
G_1	Urban integration	0.600	0.768
G_2	Land use efficiency	0.680	0.806
G_3	Regulatory compliance	0.750	0.915
I_1	Return on investment	0.620	0.765
I_3	Long-term asset value	0.580	0.710
D_1	Functional integration	0.650	0.796
C_3	Process compatibility	0.550	0.681
P_1	Transfer efficiency	0.550	0.778

In summary, the contributions of this study are as follows: (1) Establishing an isomorphic mapping framework between architectural programming processes and MARL algorithmic logic; (2) Introducing CFR-based training mechanism that enables stable evolution from individual rationality (local optima) to collective rationality (global equilibrium) in complex systems; (3) Employing HRL architecture to decouple decision-making across priority scales, providing an interpretable technical paradigm for intelligent programming in MTPs.

ACKNOWLEDGMENTS

The authors are grateful to the referees for their suggestions, as well as the experts participating in this research. This work was supported by the CSC (202506260093) and NSFC (52378034).

REFERENCES

Bansal, V. K. 2021. "Integrated Framework of BIM and GIS Applications to Support Building Lifecycle: A Move toward *nD* Modeling." *J. Archit. Eng.*, 27(4).

Brown, N., Lerer, A., Gross, S., and Sandholm, T. 2019. "Deep Counterfactual Regret Minimization." *International Conference on Machine Learning*., 97.

Gerber, D. J., Pantazis, E., and Wang, A. 2017. "A multi-agent approach for performance-based architecture: Design exploring geometry, user, and environmental agencies in façades." *Autom. Constr.*, 76: 45-58.

Gronauer, S., and Diepold, K. 2022. "Multi-agent deep reinforcement learning: a survey." *Artif. Intell. Rev.*, 55(2): 895-943.

Hu, Y., Chan, A., Le, Y., and Jin, R. Z. 2015. "From Construction Megaproject Management to Complex Project Management: Bibliographic Analysis." *J. Manage. Eng.*, 31(4).

Ivanov, D., Tang, C. S., Dolgui, A., Battini, D., and Das, A. 2021. "Researchers' perspectives on Industry 4.0: multi-disciplinary analysis and opportunities for operations management." *Int. J. Prod. Res.*, 59(7): 2055-2078.

Jin, S. 2025. "The application of collective intelligence in the construction industry: a review of the current state, challenges, and opportunities." *Archit. Eng. Des. Manag.*, 1-26.

Jin, S., and Tu, H. 2024. "Current status and research progress in architectural programming: A comparative analysis between China and other countries." *Front. Archit. Res.*, 14(2): 487-509.

Keith, A., and Ahner, D. 2021. "Counterfactual regret minimization for integrated cyber and air defense resource allocation." *Eur. J. Oper. Res.*, 292(1): 95-107.

Li, Y., Wang, X. Z., Wang, W., Zhang, Z. Y., Wang, J. S., Luo, X. F., and Xie, S. R. 2021. "Learning adversarial policy in multiple scenes environment via multi-agent reinforcement learning." *Connect. Sci.*, 33(3): 407-426.

Li, Y. K., Han, Y. L., Luo, M. X., and Zhang, Y. 2019. "Impact of Megaproject Governance on Project Performance: Dynamic Governance of the Nanning Transportation Hub in China." *J. Manage. Eng.*, 35(3).

Mok, K. Y., Shen, G. Q., and Yang, J. 2015. "Stakeholder management studies in mega construction projects: A review and future directions." *Int. J. Proj. Manag.*, 33(2): 446-457.

Oroojlooy, A., and Hajinezhad, D. 2023. "A review of cooperative multi-agent deep reinforcement learning." *Appl. Intell.*, 53(11): 13677-13722.

Pena, W. M., and Parshall, S. A. 2012. *Problem Seeking: An Architectural Programming Primer*. John Wiley and Sons. London.

Veloso, P., and Krishnamurti, R. 2023. "Spatial synthesis for architectural design as an interactive simulation with multiple agents." *Autom. Constr.*, 154.